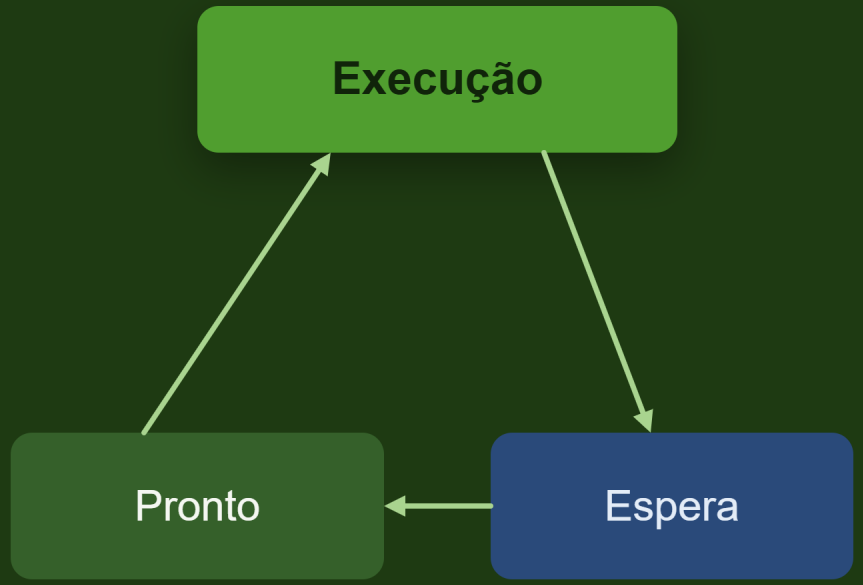
ARQUITETURA DE SISTEMAS OPERACIONAIS · CAPÍTULO 5

Processo

O que é, do que é feito, como muda de estado e como o sistema operacional o controla

Prof. Christiano Rossini · IFAL Campus Maceió



CAP. 5 · ROTEIRO

A sequência do capítulo

5.1
Introdução

5.2
Estrutura do processo

5.3
Estados do processo

5.4
Mudanças de estado

5.5
Criação e eliminação

5.6
CPU-bound e I/O-bound

5.7
Foreground e background

5.8
Formas de criação

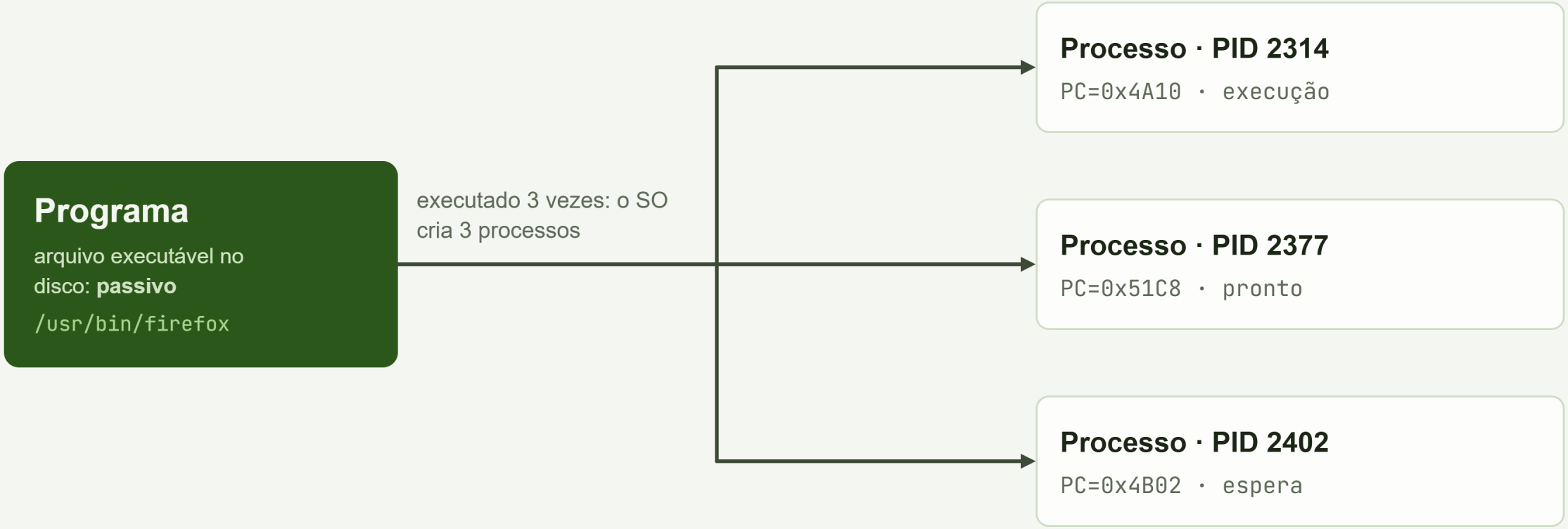
5.9
Independentes, subprocessos e threads

5.10
Processos do sistema operacional

5.11
Sinais

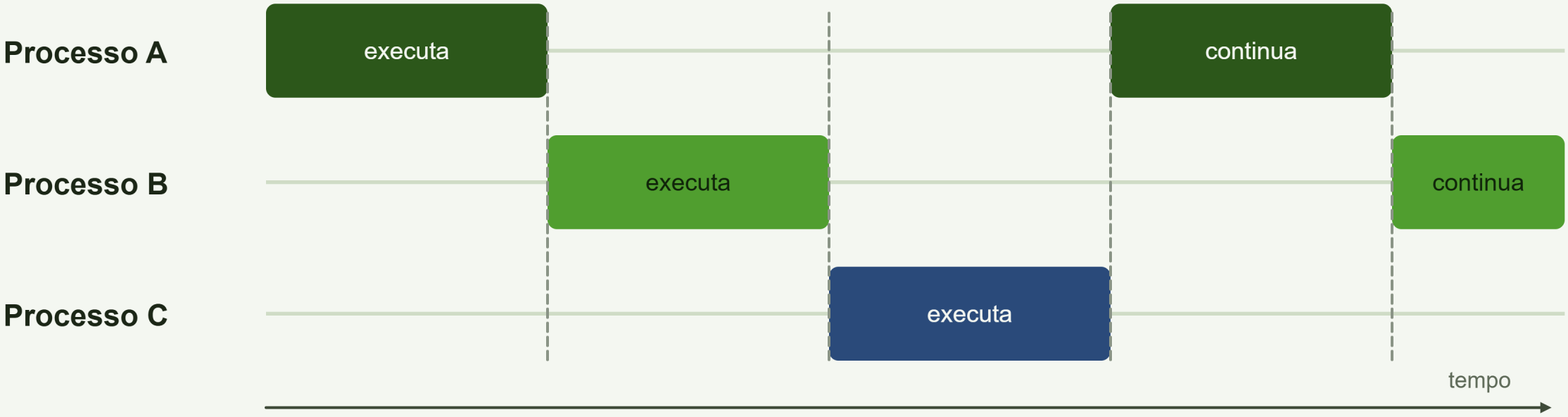
+
Prática no Linux e revisão

Programa não é processo



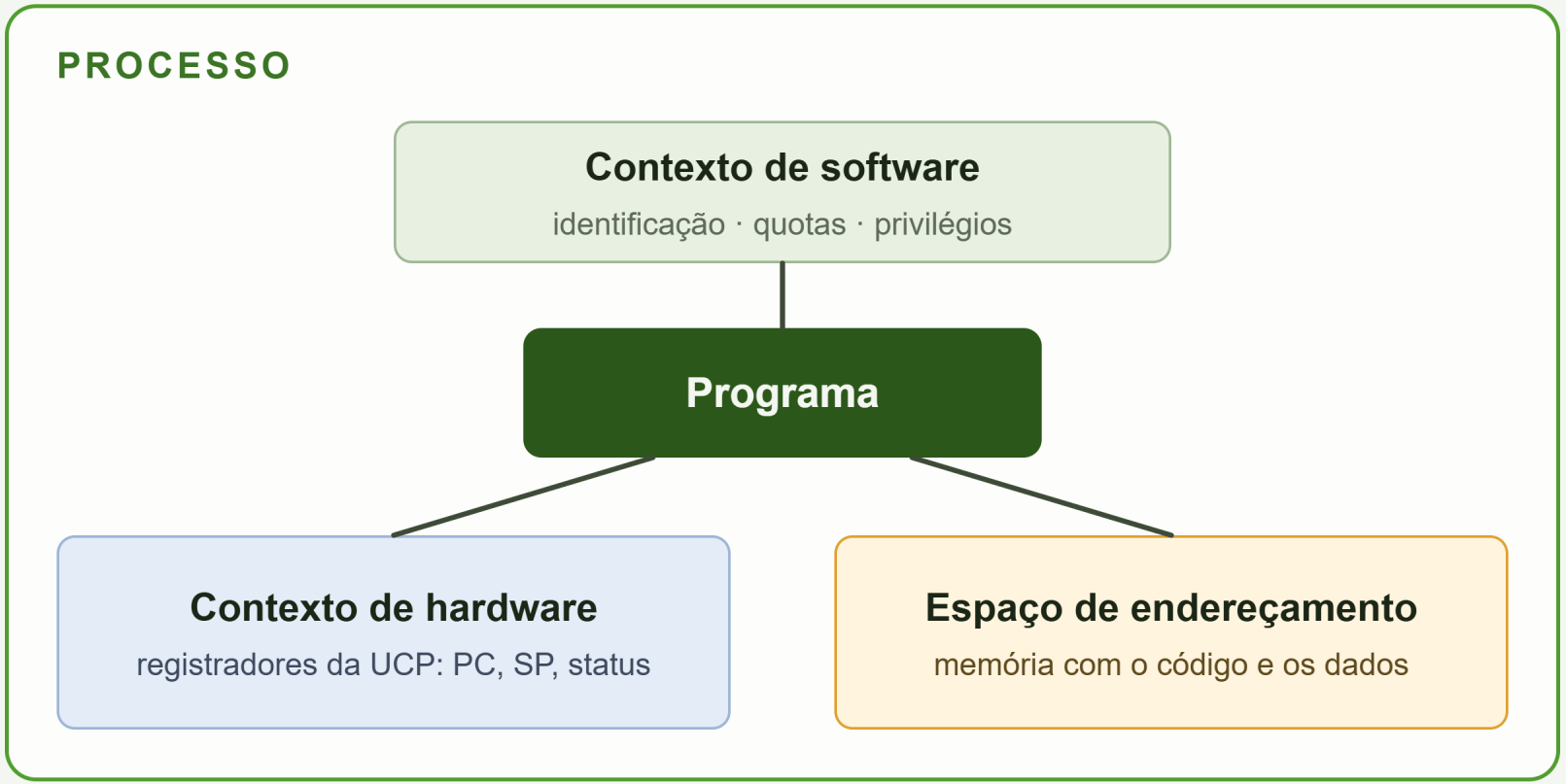
Mesmo código, três processos: cada um com **seus próprios registradores, dados e estado**.

Multiprogramação: interromper e retomar



Cada linha tracejada é uma **mudança de contexto**: o SO guarda tudo de que o processo precisa para depois continuar exatamente de onde parou.

Um processo tem três partes



Juntas, as três partes guardam tudo o que o SO precisa para **parar e retomar** o programa sem que ele perceba.

Contexto de hardware: os registradores

UCP · REGISTRADORES

PC	contador de programa: endereço da próxima instrução	`0x4A10`
SP	apontador de pilha: topo da pilha do processo	`0x7FF0`
PSW	registrador de status: modo de acesso e flags	`0x0202`
R0…Rn	registradores de uso geral: valores em cálculo	…

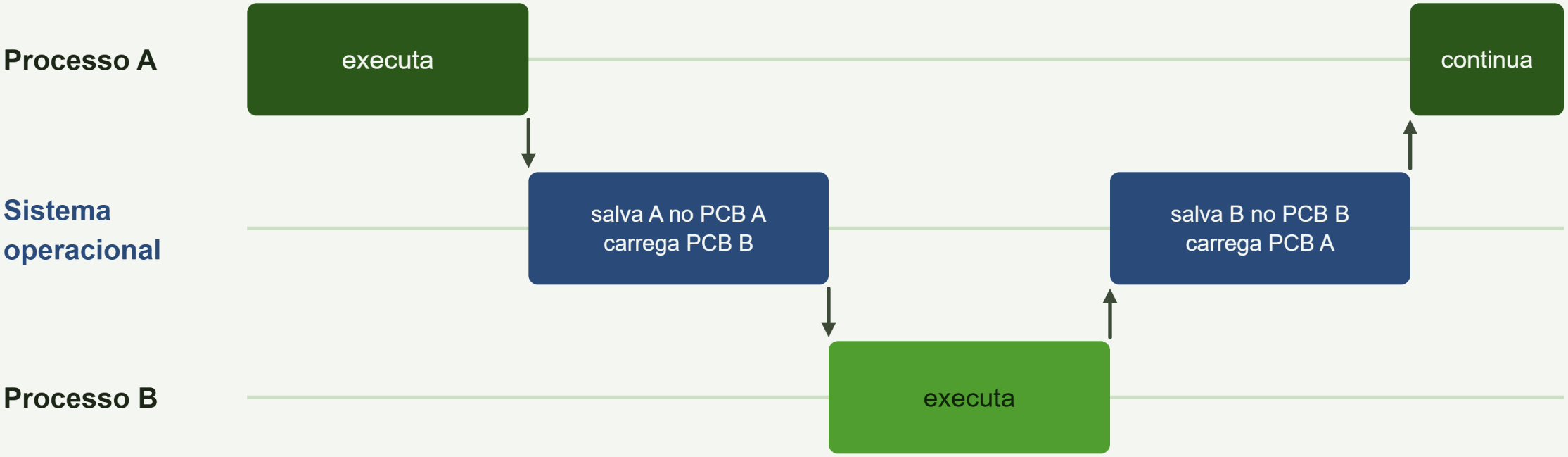
Salvar

Ao perder a UCP, o SO copia os valores dos registradores para o PCB do processo.

Restaurar

Ao voltar a executar, o SO recarrega esses valores e o programa segue da instrução em que parou.

Mudança de contexto, passo a passo



Durante a troca, nenhum processo de usuário avança: a mudança de contexto é **custo (overhead)** do sistema.

Contexto de software: quem é e o que pode



Identificação

- PID: número único do processo
- UID: usuário dono do processo
- nome do processo

Serve para o SO e outros processos referenciarem este processo.



Quotas

- máx. de arquivos abertos
- máx. de memória alocada
- máx. de operações de E/S pendentes
- máx. de subprocessos e threads

Quota insuficiente: o processo fica lento, é interrompido ou nem executa.



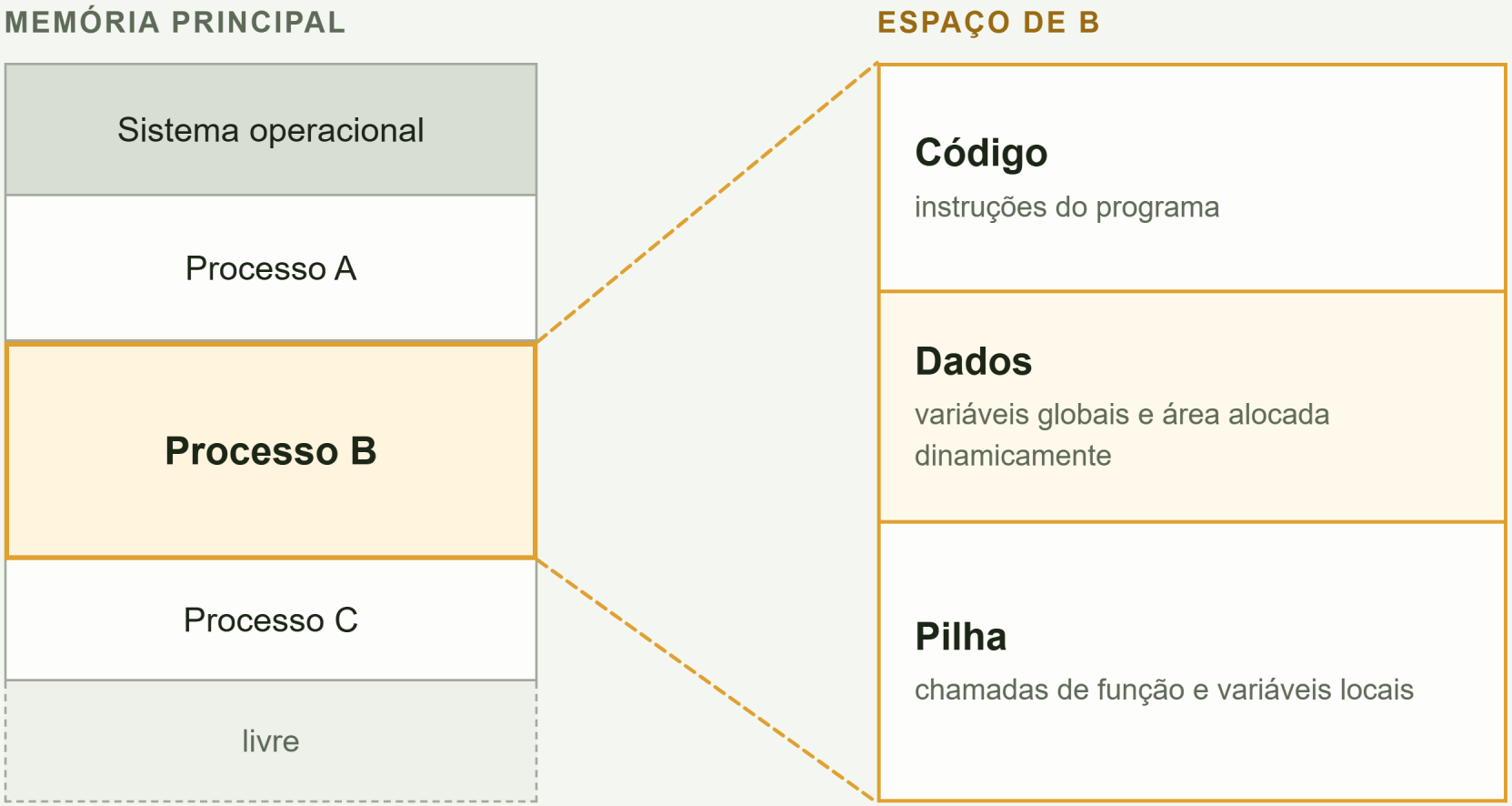
Privilégios

- sobre si: mudar a própria prioridade
- sobre outros: eliminá-los
- sobre o sistema: desligá-lo, criar usuários

Privilégios totais: administrador (root).

```
$ ps -eo pid,uid,comm → a identificação de cada processo
```

Espaço de endereçamento: a memória do processo



Protegido

Nenhum outro processo pode ler ou escrever aqui.

Mais adiante

Caps. 9 e 10: gerência de memória e memória virtual.

PCB: o processo visto pelo SO

PCB · PID 2314	
Ponteiros	listas
Estado do processo	controle
Nome (PID)	**software**
Prioridade	**software**
Registradores	**hardware**
Limites de memória	**endereçamento**
Arquivos abertos	**software**

ÁREA DE MEMÓRIA DO SO



Tudo o que o SO sabe sobre um processo está no seu **PCB**.

Criar um processo = alocar e preencher um PCB.
Eliminar = liberar o PCB.

Os ponteiros encadeiam os PCBs nas listas de cada estado.

Três estados possíveis

Execução

running

Está usando a UCP agora. Com um só núcleo, apenas um processo por vez.

No banco: sendo atendido no caixa.

Pronto

ready

Tem tudo para executar; só aguarda a sua vez de usar a UCP.

No banco: na fila, esperando o caixa.

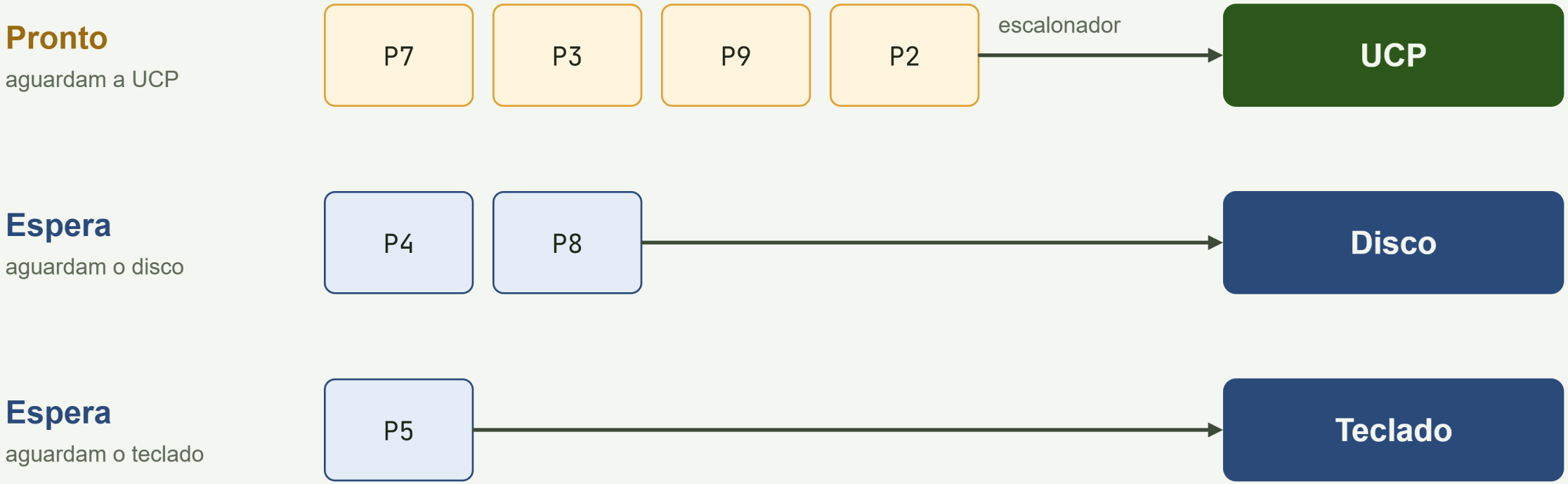
Espera

wait · bloqueado

Aguarda um evento: fim de uma E/S, um sinal, um recurso liberado.

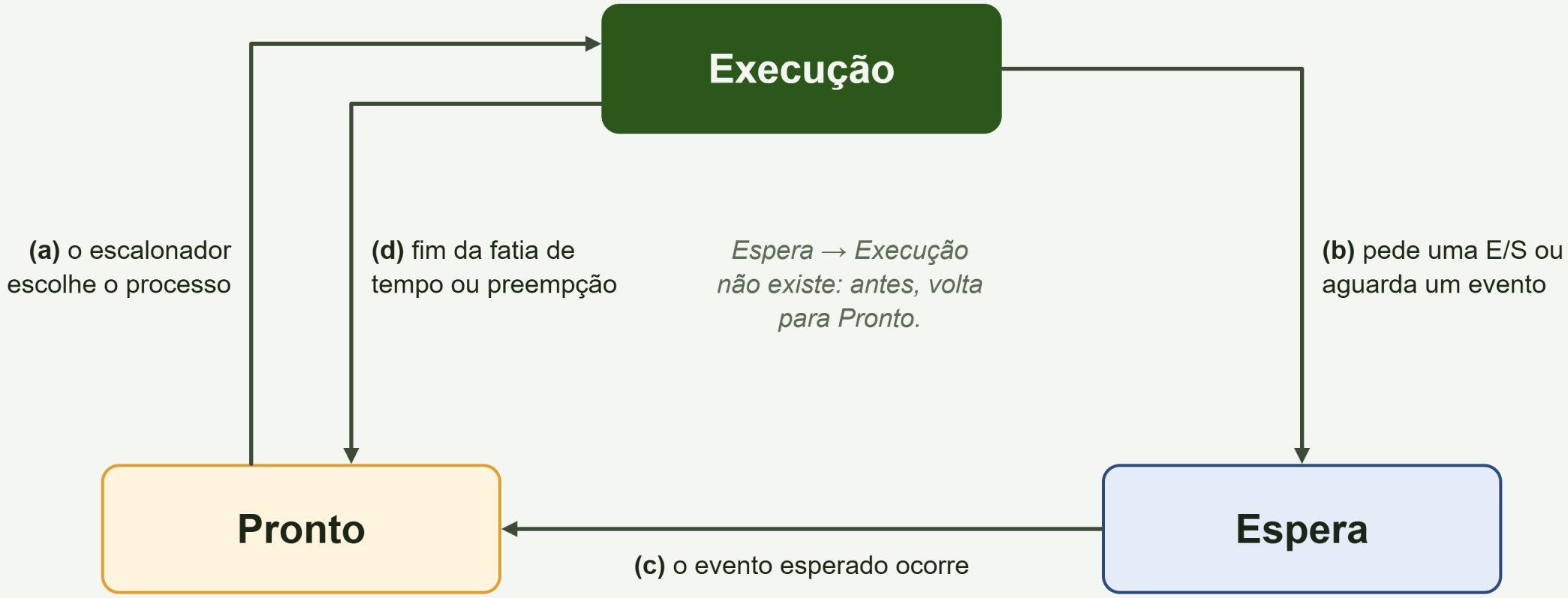
No banco: saiu da fila para buscar um documento.

Cada estado é uma lista de PCBs

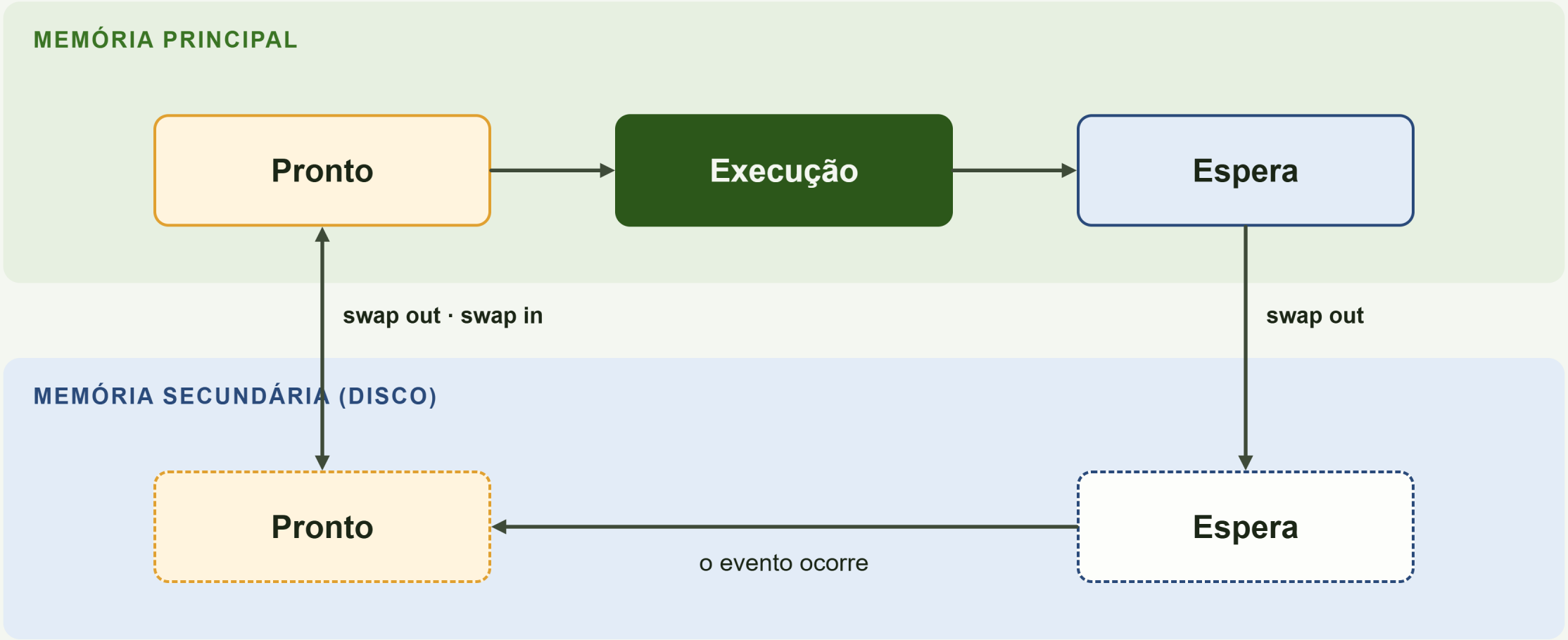


Quando a E/S termina, o PCB sai da lista de espera do dispositivo e vai para o **fim da lista de prontos**.

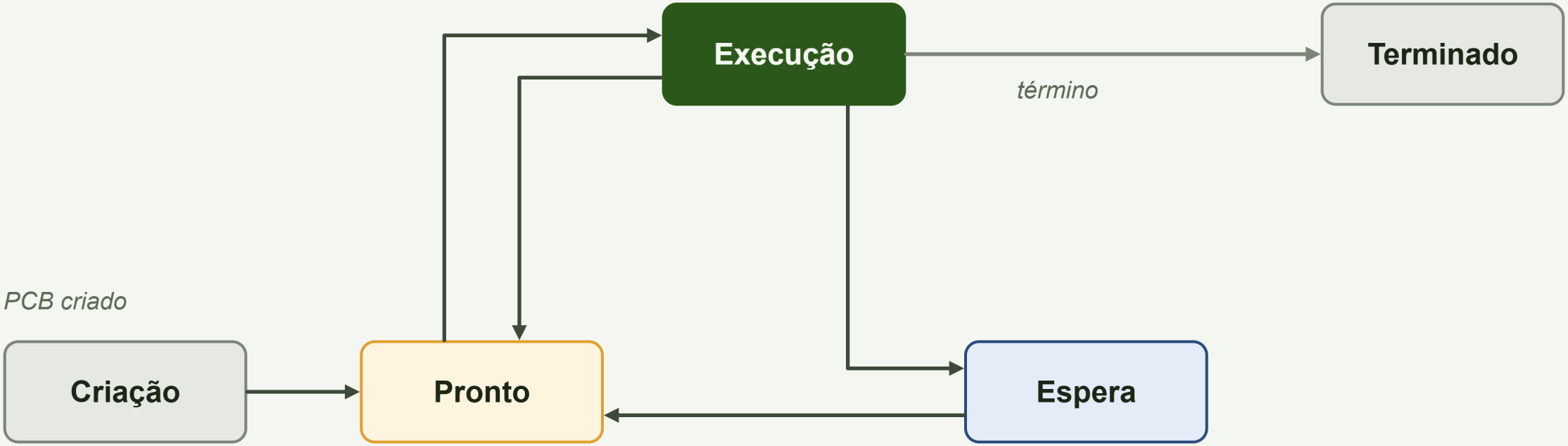
As quatro mudanças de estado



Pronto e espera também fora da memória



Mais dois estados: criação e terminado



Criação

O PCB já existe, mas o processo ainda não entrou na lista de prontos (ex.: limite de processos atingido).

Terminado

Não executa mais, mas o PCB continua em memória (ex.: contabilização) até ser liberado.

CPU-bound ou I/O-bound: onde passa o tempo?



Ligado à UCP

Passa quase todo o tempo em execução, com pouca leitura e gravação.

Ex.: cálculo científico, compressão de vídeo

Ligado à E/S

Passa quase todo o tempo em espera, aguardando disco, rede ou usuário.

Ex.: sistemas comerciais com banco de dados, editores

Foreground, background e pipe

Foreground

O usuário interage durante a execução.



entrada e saída ligadas ao terminal

Background

Não há interação com o usuário.



entrada e saída ligadas a arquivos · ex.: backup

Pipe

a saída de um processo vira a entrada de outro



Três formas de criar um processo

Logon interativo

Usuário informa login e senha

↓

SO valida e cria um processo para o usuário

↓

Logoff elimina o processo

Linguagem de comandos

Usuário digita um comando no shell

↓

Shell pede ao SO um novo processo

↓

```
$ ./soma
```

Rotina do SO

Um programa chama uma rotina do sistema

↓

SO cria o novo processo

↓

`fork()` · `CreateProcess()`

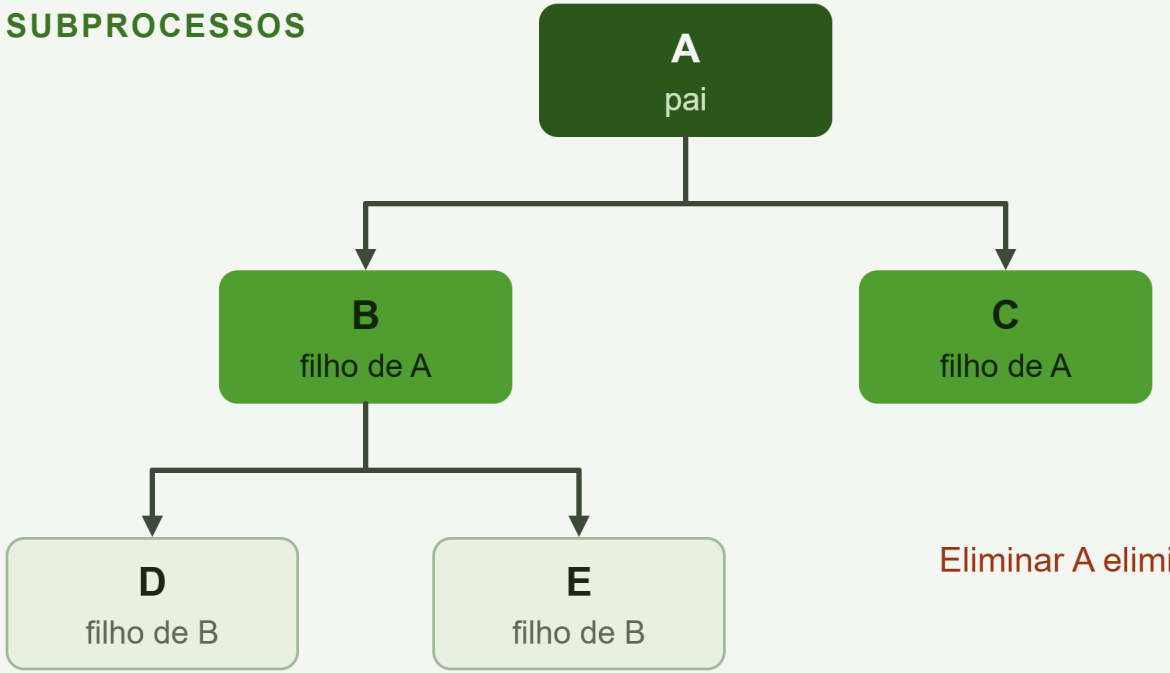
Independentes ou em hierarquia

INDEPENDENTES

Processo X **Processo Y**

Sem vínculo: cada um com seus próprios recursos e quotas.

SUBPROCESSOS



Eliminar A elimina B, C, D e E.

Subprocessos dependem do pai

Podem compartilhar as quotas do pai e são eliminados junto com ele.

Mas custam caro

Cada um tem contexto e espaço próprios: criar e trocar consome tempo e memória.

Threads: vários fluxos, um só processo

2 subprocessos

Filho 1

- Contexto de software
- Contexto de hardware
- Espaço de endereçamento

Filho 2

- Contexto de software
- Contexto de hardware
- Espaço de endereçamento

Tudo duplicado: mais memória, trocas mais lentas.

1 processo com 3 threads

Contexto de software · compartilhado

Thread 1
contexto de hardware próprio

Thread 2
contexto de hardware próprio

Thread 3
contexto de hardware próprio

Espaço de endereçamento · compartilhado

Criar, trocar e comunicar fica mais barato. Detalhes no cap. 6.

O próprio SO também roda como processos

Linux: *daemons* · Windows: *serviços*. Núcleo menor e mais estável, mas cada serviço consome recursos.

Auditoria e segurança
registra acessos e eventos

Serviços de rede
servidor web, acesso remoto

Contabilização de recursos
uso de UCP e memória

Contabilização de erros
falhas de hardware e software

Gerência de impressão
fila de impressão (spooling)

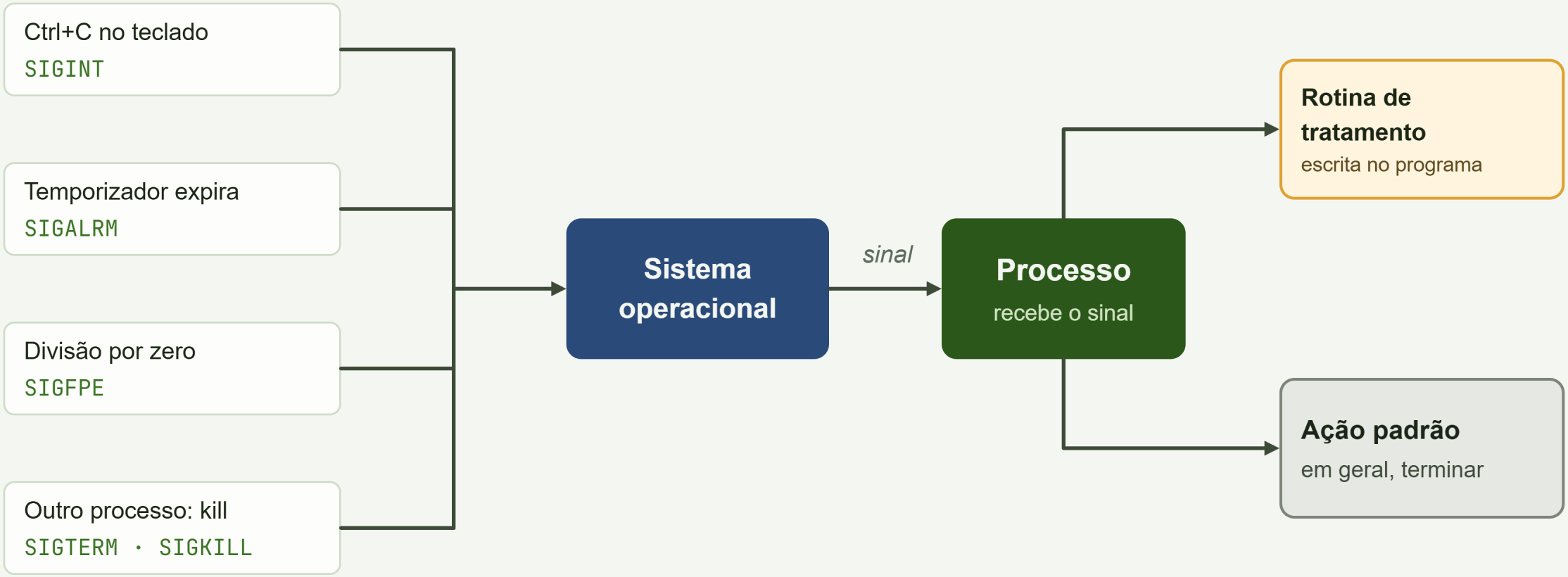
Gerência de jobs batch
lotes sem interação

Temporização
tarefas agendadas (cron)

Comunicação de eventos
avisa processos de ocorrências

Interface de comandos
o shell: bash, PowerShell

Sinais: o SO avisa o processo



O sinal está para o processo assim como a **interrupção** está para o SO: um aviso assíncrono de um evento.

Vendo o capítulo no terminal Linux

Conceito	Comando	O que observar
Identificação	`ps -eo pid,uid,comm`	PID e UID de cada processo
Estados	`ps -eo pid,stat,comm`	R execução/pronto · S, D espera · Z terminado
Subprocessos	`pstree -p`	a árvore pai → filhos
Background	`sleep 60 & jobs fg`	o shell fica livre com &
Pipe	`ps -ef | grep bash`	saída de um, entrada do outro
Sinais	`kill -INT PID · kill -9 PID`	pedir para parar × forçar
CPU × I/O-bound	`top`	coluna %CPU de cada processo

CAP. 5 · RESUMO

O capítulo em um quadro

Processo = programa em execução + o contexto que permite ao SO **pará-lo e retomá-lo**.

Estrutura

- contexto de hardware
- contexto de software
- espaço de endereçamento
- tudo reunido no PCB

Estados

- execução, pronto, espera
- quatro transições
- swapping
- criação e terminado

Classificações

- CPU-bound × I/O-bound
- foreground × background
- independentes, subprocessos, threads

SO e processos

- três formas de criação
- processos do sistema
- sinais

Para revisar

1. Por que o conceito de processo é essencial em um sistema multiprogramável?
2. O que fica no contexto de hardware, no de software e no espaço de endereçamento?
3. Por que um processo não passa diretamente de espera para execução?
4. Dê um exemplo de processo CPU-bound e um de I/O-bound. Qual fica mais tempo em espera?
5. O que subprocessos e threads compartilham — e o que não compartilham?